

DEPARTMENT OF COMPUTER SCIENCE

FINAL PROJECT REPORT

Pac-Man Capture the Flag

CSE 140 ARTIFICIAL INTELLIGENCE

Strategy & Implementation

Team: TurkiyeIsTheBest

Adan Shafei
Kaan Koç
Aleksandar Videv

December 8, 2025

Abstract

This report details the design, implementation, and evaluation of an autonomous agent team for the Pac-Man Capture the Flag tournament. Our approach utilizes a feature-based evaluation model with manually tuned weights to balance offensive aggression and defensive stability. The final agents demonstrate reactive decision-making capable of efficiently collecting food and intercepting invaders. While our agents achieved a strong 9/10 average performance against the baseline autograder, the final tournament score of 38/56 highlights specific areas for future improvement in lookahead search and probabilistic tracking.

1 Introduction

Our team developed a pair of agents for the Capture the Flag contest in CSE 140. The project required the design of an offensive agent that collects food and returns safely, and a defensive agent that protects our territory from invaders. We collaborated throughout development using a shared GitHub repository, Zoom meetings, and continuous communication to evaluate many versions of our agents and to merge the strongest ideas into the final submission.

2 Fundamental Problems

2.1 Offensive Challenges

The offensive agent was required to solve several dynamic pathfinding and risk-assessment problems:

- **Navigation:** Moving safely into enemy territory while minimizing path cost.
- **Resource Collection:** Efficiently gathering food within the strict time limits.
- **Threat Avoidance:** Evading non-scared ghosts that possess the ability to eliminate the agent.
- **Return Logic:** Deciding the optimal moment to return home to secure collected points.
- **Risk/Reward Trade-offs:** Balancing the potential score gain against the probability of death.

2.2 Defensive Challenges

The defensive agent focused on territorial control:

- **Invader Detection:** Identifying opponents entering our home territory.
- **Threat Assessment:** Determining which invader posed the greatest risk to our food supply.
- **Asset Protection:** Guarding food clusters effectively.
- **Patrol Stability:** Maintaining a patrol pattern that covers key chokepoints without stalling.

3 Modeling the Problems

We utilized a **linear feature-based evaluation model** for both agents. In this framework, every available action a from state s is evaluated as:

$$V(s, a) = \sum_{i=1}^n w_i \cdot f_i(s') \quad (1)$$

where w_i is the weight of the i -th feature and $f_i(s')$ is the value of that feature in the successor state s' . This allowed us to encode domain knowledge—such as distance to food, distance to invaders, and safety status—directly into the decision-making process.

3.1 Agent Models

- **Offensive Model:** Represented urgency to return, proximity to threats, opportunities to chase scared ghosts, and the value of food relative to time.
- **Defensive Model:** Represented the number/distance of invaders, the strategic value of staying on the home side, and invader proximity to defended food.

4 Computational Strategy

4.1 Strategy for Offense

The offensive agent extends the standard feature extractor with custom logic to handle complex game states:

- **Dynamic Return Rule:** We implemented a hard-coded decision rule to override greedy food collection when specific thresholds were met. The agent triggers a "Return Home" mode if:

$$(\text{food} \geq 4) \vee (\text{food} \geq 3 \wedge \text{score} > 1) \vee (\text{food} \geq 3 \wedge \text{ghost_dist} < 5)$$

- **Weight Adjustment:** During the "Return Home" phase, the weight for `on_home_side` is dynamically boosted by a factor of 6.5, forcing the agent to prioritize safety over new food.
- **Aggressive Scared Ghost Pursuit:** When ghosts are scared, we apply a negative weight dependent on distance ($\approx -8.0 \times \text{dist}$), encouraging the agent to eliminate opponents for extra points.

4.2 Strategy for Defense

The defensive agent employs a heuristic to maximize interception efficiency:

- **Threat Scoring:** Invaders are ranked by a threat metric:

$$\text{threat} = \text{dist}(\text{me}, \text{invader}) - 2 \times \text{dist}(\text{invader}, \text{my_food})$$

This formulation prioritizes invaders that are closer to our food, rather than just closer to our agent.

- **Oscillation Prevention:** To prevent the agent from getting stuck in loops, we applied heavy penalties for stopped ($w = -200.0$) and reverse ($w = -3.0$).

5 Algorithmic Choices and Feature Tuning

5.1 Feature Weight Tuning

A significant portion of our development time was dedicated to manually tuning the weight vectors. We aimed for reactive agents that did not require computationally expensive search algorithms (like Minimax) during the 3-second action window.

Table 1 outlines the final weights used in our submission.

Feature Name	Offensive Weight	Defensive Weight
score	110.0	–
distance_to_food	–3.5 (0.0 when returning)	–
stopped	–150.0	–200.0
reverse	–5.0	–3.0
on_home_side	–18.0 (Boosted $6.5\times$)	120.0
distance_to_ghost	2.5	–
chase_scared	Conditional ($\approx -8.0 \times d$)	–
num_invaders	–	–2000.0
distance_to_invader	–	Conditional ($\approx -35.0 \times \text{threat}$)

Table 1: Final Feature Weights for Offensive and Defensive Agents.

5.2 Key Design Decisions

1. **Reactive Architecture:** We chose a purely reactive architecture based on successor states. This ensured our agents never timed out, even in complex board states.
2. **Simplified Cluster Analysis:** For the defensive agent, we simplified food cluster checks to only consider the nearest three food pellets. This approximation significantly reduced computation time while retaining 90% of the strategic benefit.
3. **Avoidance of Search Trees:** We deliberately avoided deep search trees (MCTS or Alpha-Beta) to minimize the risk of time-limit forfeits, relying instead on highly tuned heuristics.

6 Obstacles Encountered

- **Aggression vs. Survival:** Finding the "sweet spot" for the ghost distance weight was difficult. If too high, the agent was too timid; if too low, it died often.
- **Local Optima:** The defensive agent occasionally got stuck in patrol loops. We mitigated this by increasing the penalty for stopping and reversing.
- **Overfitting:** We found that weights tuned perfectly for specific maps (like capture-default) sometimes performed poorly on randomized layouts.

7 Evaluation and Results

7.1 Quantitative Performance

Our final project submission achieved a total score of **38/56**. This score is a composite of our autograder performance and our tournament ranking.

- **Baseline Performance:** We achieved a strong **9/10 average** against the autograder's baseline model. This confirms that our foundational logic—collecting food and returning—was robust and correctly implemented.
- **Tournament Ranking:** While we consistently beat the baseline, our performance in the wider tournament was mixed. We successfully defeated weaker agents but struggled against teams that likely implemented probabilistic tracking or deeper lookahead search.

7.2 Qualitative Analysis

Visual inspection of replays showed that the Offensive Agent was highly efficient at pathing but occasionally failed to anticipate "traps" set by enemy agents around corners (a limitation of zero-lookahead). The Defensive Agent was excellent at chasing but could be lured away from the food by a decoy invader.

8 Conclusion

The Capture the Flag project provided a practical environment to apply AI concepts such as heuristic evaluation and adversarial reasoning. Our team successfully implemented a pair of agents that reliably outperformed the baseline, achieving a 9/10 average in preliminary checks. While the final tournament score of 38/56 suggests room for improvement against advanced adversaries, the project demonstrated the effectiveness of feature-based reactive control in real-time environments.